A Low-Cost Closed-Loop MLOps Pipeline with AutoML and Multi-Signal Drift Alerting on Serverless Cloud

Prajna
School of CSE
RV University
Bengaluru, India
1RVU23CSE339

Nandan Kumar S
School of CSE
RV University
Bengaluru, India
1RVU23CSE299

Dr. Revansiddappa
Associate Professor
School of CSE
RV University
Bengaluru, India

***Abstract*—Deployed machine-learning models degrade silently as data shifts, yet most pipelines either retrain on a fixed schedule or alert on raw statistical tests that fire on harmless shifts. We present an end-to-end, open-source MLOps loop that joins AutoML model selection (FLAML), a FastAPI prediction service designed for AWS Lambda, an S3-backed model registry and inference log, a scheduled drift job, and CI/CD-triggered retraining with a deployment gate and zero-downtime hot-swap. Its core is a model-agnostic, label-free *multi-signal* drift detector that raises a retrain alarm only when at least two of three signals—input-feature shift, prediction-distribution shift, and confidence drop—agree over consecutive windows. In 20-seed controlled experiments, single-test KS and PSI detectors alarmed on 100% of windows after a benign shift in features the model ignores, while the multi-signal detector alarmed on 2%, and still caught 90–95% of harmful shifts that cut accuracy by 8–17 points within about one window. On the real Elec2 electricity stream, a static model fell from 83.6% hold-out accuracy to 67.8%; retraining triggered by our detector recovered 74.4%, matching KS- and PSI-triggered retraining within run-to-run variation while using 19% fewer retrains. A live HTTP replay of 20,000 records ran nine detect–retrain cycles and six gated redeployments without a restart, with a median of 6.5 s from alarm to serving and 76.2% accuracy against 68.0% for the static model. Deployed on AWS (Lambda, API Gateway, S3, CloudWatch) with one SAM template, the same loop detected drift in live traffic, retrained inside Lambda in 13 s and promoted a new model (83.0% vs. 78.8%) that the API served without a redeploy. Lambda and CloudWatch use stays within AWS always-free allowances.**

***Index Terms*—MLOps, AutoML, concept drift, data drift, model monitoring, serverless, AWS Lambda, CI/CD, continuous training**

I. Introduction

A model in production is a living system: it must be served, versioned, watched and replaced as the data it sees moves away from the data it was trained on [1]. Vela *et al.* found temporal degradation in 91% of 128 model–dataset pairs across four industries [2]. Because labels usually arrive late, degradation must be detected from inputs and predictions alone, and the response—retraining and redeployment—should be automatic [3].

Our literature review of ten recent works exposed three gaps. *(G1)* No surveyed system closes the full loop of AutoML selection, live serving, drift detection, alerting and CI/CD redeployment within a free-tier budget; existing options are paid managed services [4] or isolated components [5]–[7]. *(G2)* Statistical detectors flag every significant shift, including shifts that do not affect accuracy [8], while uncertainty-based detectors require a specific model family [7]. *(G3)* Few open experiments measure detection behaviour and recovery on a live prediction API.

Contributions. (1) An open-source closed-loop pipeline—AutoML, serving, logging, drift job, gated retraining, hot-swap—with a validated AWS SAM template and GitHub Actions workflows that keep the cloud footprint within always-free limits. (2) A model-agnostic, label-free multi-signal detector with voting and persistence. (3) A reproducible evaluation on controlled drift with ground truth, the real Elec2 stream, and a live HTTP service.

II. Related Work

MLOps. Kreuzberger *et al.* [3] define MLOps around CI/CD, versioning, continuous training and monitoring; a multivocal review of 198 sources [9] reports that cost and complexity push small teams back to manual workflows. Garg *et al.* [5] map GitOps push/pull deployment onto ML but do not address drift-triggered retraining. **AutoML.** AutoML solves the combined algorithm selection and hyper-parameter optimisation (CASH) problem [10]. A benchmark on 73 datasets found all frameworks beat a random-forest baseline with at most 2.2% spread among them, but none considers deployment or lifecycle [6]. We use FLAML for its cost-frugal search and warm-start support [11]. **Drift detection.** Error-rate detectors such as ADWIN [12] need labels [13]. Two-sample tests on inputs are label-free but sensitive to irrelevant shift [14]: Eck *et al.* observed significant feature drift with stable accuracy in a deployed supply-chain system [8]. Uncertainty Drift Detection [7] suppresses such alarms but needs a dropout neural network. **Monitoring and retraining.** SageMaker Model Monitor [4] detects several drift types at

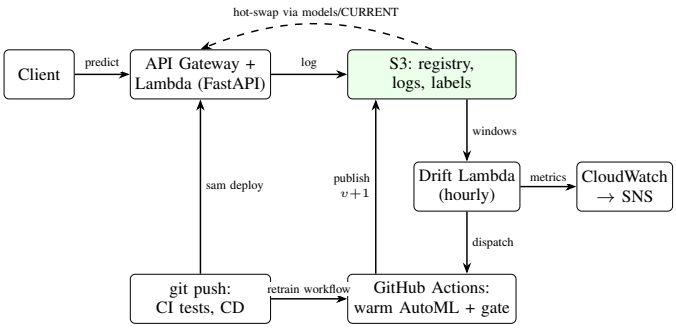


Fig. 1. Closed-loop architecture. Models are data in S3; a new version reaches the running API through an atomic pointer swap, with no redeploy.

scale but stops at the alert and is a paid service. Katalay *et al.* [15] combine statistical criteria to trigger retraining of neural classifiers, evaluated offline on anomaly-detection benchmarks.

III. System Design

Fig. 1 shows the loop. Every component reads and writes through one storage interface with a local-directory and an S3 back end, so experiments and the cloud deployment share the same code.

AutoML. FLAML searches LightGBM, XGBoost, random forest, extra trees and ℓ_1 logistic regression with a fixed time budget and a time-ordered hold-out. Retraining is *warm-started* from the previous best configuration of each learner, which lets a short budget (6–8 s) give a competitive model.

Serving and logging. A FastAPI service exposes `/predict`, `/feedback` (delayed labels) and `/health`. It runs under uvicorn locally and on Lambda via Mangum in a container image. Each request is logged to S3 with a globally ordered id. The service re-reads the registry pointer `models/CURRENT` every R seconds and hot-swaps to a newer version; because the pointer is written last, readers never see a half-written version.

Multi-signal drift detection. For a reference window $\mathcal{R}$ (held-out data and the serving model's predictions on it) and a live window $\mathcal{W}$ of $n{=}500$ records with d features, we compute three label-free signals:

- S_{in}: the share of features that are shifted under *both* a Bonferroni-corrected KS test ($p < 0.05/d$) and PSI > 0.2 is at least $1/4$, where PSI $= \sum_k (c_k - r_k)\ln(c_k/r_k)$ over ten reference-quantile bins.
- S_{pred}: PSI of the predicted-probability distribution exceeds 0.1.
- S_{conf}: mean confidence $|2\hat{p}-1|$ drops by more than 0.05.

A window votes "drift" when at least two signals fire; an alarm needs two consecutive drift votes, followed by a two-window cooldown. Since S_{in} alone can never alarm, shifts in features the model does not use are suppressed, while shifts that move the model's outputs are caught. The rule uses only inputs and outputs, so it applies to whatever model AutoML selects.

Retraining and deployment gate. On alarm, the drift Lambda retrains in place; optionally it can instead send a GitHub `repository_dispatch` so that a GitHub Actions workflow retrains. Either path takes the 3,000 most recent labelled records, runs warm-started AutoML on the older 80%, and promotes the candidate only if it matches or beats the serving model on the newest 20%. The winner is refit on all fresh data and published as version $v{+}1$ together with a new out-of-sample reference window. CI runs unit and end-to-end tests plus `cfn-lint`; CD builds the image and runs `sam deploy` using GitHub OIDC, so no long-lived keys are stored.

Cost. The design uses two Lambda functions, one HTTP API, one S3 bucket, three custom metrics and one alarm. AWS's always-free tier covers 1M Lambda requests and 400,000 GB-s per month and 10 custom CloudWatch metrics and alarms [16]; an hourly drift job at 2 GB for 20 s uses about 28,800 GB-s per month. Both functions ship as one container image; using the CPU-only XGBoost build keeps the CUDA libraries (about 500 MB) out of it. S3, API Gateway and ECR are billed against the sign-up credits AWS gives accounts opened after July 2025.

IV. Experimental Setup

Data. *Controlled:* a synthetic tabular task (`make_classification`, 14,000 rows, 8 features of which 4 are informative and 4 are pure noise), so the effect of every shift is known. *Real:* Elec2 [17], 45,312 half-hourly records of the Australian NSW electricity market, label = price up/down, with 7 features (date index dropped). Elec2 is a standard real-world drift benchmark [13].

Detectors. KS: Bonferroni-corrected per-feature KS test (practice in [8]). PSI: any feature PSI > 0.2. MS: ours. Ablations: MS with one vote (logical OR) and without persistence. ErrorRate: alarm when window accuracy falls 5 points below the post-training value; it needs true labels at check time, so it is a supervised reference, not a deployable label-free option.

Exp. 1 (controlled, 20 seeds). Per seed, AutoML (8 s) trains on 3,000 rows; 1,000 held-out rows form the reference; 30 windows of 300 rows follow, with drift injected from window 15: *benign* (all four noise features shifted by 1σ or 2σ), *harmful* (two informative features shifted by 0.3, 0.6 or 1.0σ), or *concept* (30% of labels flipped, $P(X)$ unchanged). No retraining, to isolate detection.

Exp. 2 (Elec2, 3 runs). Initial AutoML (30 s) on rows 0–2,999, reference on 3,000–3,999; the next 41,000 rows arrive in 82 windows of 500 (test-then-train). Labels lag by one window. Every strategy uses the same warm-started retrain (6 s budget, 3,000 freshest labels) and gate, plus a two-window cooldown for triggered strategies.

Exp. 3 (live service). The FastAPI service runs under uvicorn; 20,000 Elec2 rows (4,000–23,999) are sent as 400 HTTP requests of 50 rows, labels are posted to `/feedback` one window late, and the drift job runs every 500 rows, retrains in-process and publishes to the registry while the server keeps running. Hardware: a 2-vCPU Linux container; Python 3.13, FLAML 2.7, LightGBM 4.7, XGBoost 2.1.

TABLE I

CONTROLLED DRIFT, 20 SEEDS: SHARE OF POST-DRIFT WINDOWS ALARMED (BENIGN: LOWER IS BETTER; HARMFUL: HIGHER IS BETTER). ACCURACY CHANGE AFTER DRIFT IN BRACKETS.

	Benign		Harmful		
Detector	1.0σ (-0.1)	2.0σ (-0.3)	0.3σ (-2.2)	0.6σ (-7.8)	1.0σ (-16.6)
KS	1.00	1.00	0.99	1.00	1.00
PSI	1.00	1.00	0.41	1.00	1.00
MS 1-vote (OR)	0.93	0.93	0.27	0.93	0.93
MS no persistence	0.10	0.47	0.05	0.86	0.95
MS (full)	**0.02**	**0.36**	0.02	0.76	0.88
ErrorRate (labels)	0.00	0.00	0.09	0.64	0.91

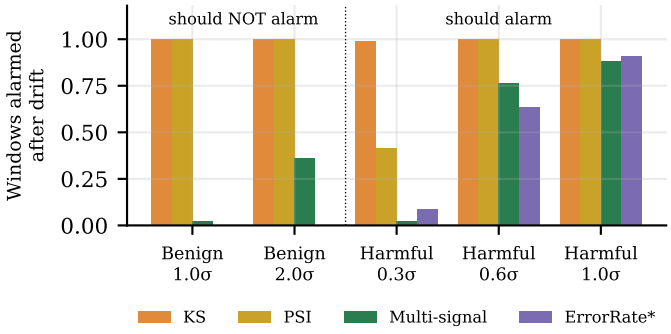


Fig. 2. Share of post-drift windows alarmed, mean of 20 seeds. KS and PSI cannot tell benign from harmful shift; MS can.

TABLE II

ELEC2 STREAM, MEAN OVER 3 RUNS (82 WINDOWS OF 500). RETRAINS AND PROMOTIONS ARE COUNTS PER RUN.

Strategy	Acc. (%)	Worst win.	Retrains	Promoted
No retraining	67.8±0.6	51.1	0.0	0.0
Fixed (every 4 win)	73.9±0.5	52.9	20.0	15.7
Fixed (every 10 win)	73.4±0.6	53.8	8.0	5.0
KS-triggered	74.7±0.4	56.3	28.0	21.7
PSI-triggered	74.6±0.3	54.4	28.0	22.0
Multi-signal (ours)	74.4±0.3	55.0	22.7	17.0
ErrorRate (labels)*	73.3±0.6	56.4	17.3	14.0

*needs true labels at check time (supervised reference).



Fig. 3. Elec2 accuracy per 500-record window (one run). Ticks at the bottom mark retrains.

V. RESULTS

A. Benign versus harmful drift (Exp. 1)

Table I and Fig. 2 give the main result. In the no-drift runs, MS raised no alarm in any of the 600 windows, whereas KS raised at least one false alarm in 7 of 20 runs. After a benign 1σ shift that changed accuracy by -0.1 points, KS and PSI alarmed on every window, as [8] observed in production; MS alarmed on 2%. For harmful shifts of 0.6σ and 1.0σ (accuracy -7.8 and -16.6 points), MS alarmed in 90% and 95% of runs with a mean first-alarm delay of 1.2 and 1.0 windows—the minimum the persistence rule allows. The ablations show that voting does most of the work (OR-voting alarms on 93% of benign windows) and persistence removes most of the remainder.

Three limits are visible. (i) At 2σ benign shift MS alarmed on 36% of windows: tree ensembles split slightly on noise features, so very large shifts in them do move predictions. (ii) A 0.3σ harmful shift (-2.2 points) is mostly missed; the label-based reference also alarms on only 9% of these windows, as the drop is below its 5-point threshold. (iii) Pure concept drift, where $P(X)$ is unchanged, is invisible to every label-free detector (0% detection) while ErrorRate caught all 20 runs; delayed labels remain necessary for this case.

B. Real-world drift on Elec2 (Exp. 2)

Table II and Fig. 3 show strong real-world degradation: the initial AutoML model scored 83.6% on its hold-out window but only 67.8% on the stream. Every retraining strategy recovered 5.5–6.9 points. KS and PSI fired on every window their cooldown allowed (28 of a possible 28 retrains), so on Elec2 they behave like a fixed schedule of every third window. MS reached 74.4%, within 0.3 points of KS and PSI—inside the ±0.3–0.4 run-to-run spread—while using 19% fewer retrains (22.7 versus 28), and it was 0.5 points above a fixed schedule with a similar retrain count (every 4 windows: 73.9% with 20 retrains). The cheapest schedule (every 10 windows) lost 1.0 point against MS. The label-based ErrorRate reference retrained less often (17.3) but scored lower, because its fixed 5-point threshold reacts only after accuracy has already dropped. At a 6 s AutoML budget one retrain took about 7 s of CPU.

C. Live closed loop (Exp. 3)

Table III shows the loop running end to end on a live service. Each of the 9 alarms led to a retrain; the gate rejected 3 candidates that were not better than the serving model, and 6 new versions were hot-swapped into the running server without a restart or a failed request. Time from alarm to serving was dominated by the 6 s AutoML budget; the pointer swap took under 40 ms. AutoML picked XGBoost, LightGBM and extra trees at different retrains, which is why a detector tied to one model family [7] would not fit this loop. Over the same 20,000 requests the closed loop scored 76.2% against 68.0% for the static initial model.

TABLE III
LIVE HTTP REPLAY OF 20,000 ELEC2 REQUESTS.

Requests (50 rows each)	400
Latency p50 / p95 / p99 (ms)	7.0 / 21.3 / 25.2
Drift windows checked / alarms	40 / 9
Retrains promoted by gate	6 of 9
Alarm→serving, median (max)	6.5 s (11.1 s)
detect / retrain / swap (s)	0.03–0.12 / 6.2–10.9 / <0.04
Accuracy, static v1 (same traffic)	68.0%
Accuracy, closed loop	**76.2%**

TABLE IV
CLOSED LOOP DEPLOYED ON AWS (AP-SOUTH-1) WITH ONE SAM STACK.

Seed v1 (AutoML in Lambda, 30 s budget)	Extra Trees
Traffic via API Gateway (rows 4,000–6,999)	3,000 rows
Drift windows checked / alarms	5 / 1
Retrain in Lambda (warm AutoML + gate)	13.0 s
Candidate vs. serving, newest 20%	83.0% vs. 78.8%
Result	v2 (XGBoost) promoted
Next 3,000 requests served by	v2, no redeploy

D. Deployment on AWS (Exp. 4)

Table IV repeats the loop on AWS. One `sam deploy` created the S3 bucket, both Lambda functions, the HTTP API, the hourly schedule, the CloudWatch alarm and the SNS topic. Model v1 was trained by invoking the drift function, and 3,000 Elec2 rows with their labels were sent through API Gateway. The next drift check raised an alarm. The function then retrained in place and the gate promoted an XGBoost model, so AutoML changed the model family on its own. The API served every following request from v2. The detector logic was identical to the local runs, while storage, scheduling and metrics were AWS services.

VI. DISCUSSION AND LIMITATIONS

The experiments support the design goal of MS: alarm when the model's behaviour changes, not whenever a feature moves. On data where features drift heavily but harmlessly, this removes the retrain storms that KS and PSI cause. On Elec2, where most drift is harmful, all triggered strategies end close together and the gain of MS is efficiency rather than accuracy. Limitations: concept drift with unchanged inputs needs labels, so a deployment should combine MS with a delayed-label error check; thresholds were set once, not tuned per dataset; AutoML with a time budget is not fully deterministic; and the AWS deployment was a short trial (6,000 requests), so Lambda cold-start latency and long-term cost were not measured.

VII. CONCLUSION

We built and evaluated a closed-loop MLOps pipeline that joins AutoML, serverless serving, drift monitoring and gated CI/CD retraining at near-zero cloud cost, addressing gaps G1–G3. Its multi-signal detector cut false alarms on benign drift from 100% (KS, PSI) to 2% of windows while still catching harmful drift within about one window. A trial on AWS confirmed that the whole loop runs serverlessly. Future work: adding a delayed-label error signal as a fourth vote, adaptive thresholds, and a multi-week AWS deployment with real traffic.

REFERENCES

[1] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Proc. NeurIPS*, 2015.

[2] D. Vela, A. Sharp, R. Zhang, T. Nguyen, A. Hoang, and O. S. Pianykh, "Temporal quality degradation in AI models," *Scientific Reports*, vol. 12, p. 11654, 2022.

[3] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.

[4] D. Nigenda *et al.*, "Amazon SageMaker model monitor: A system for real-time insights into deployed machine learning models," in *Proc. ACM SIGKDD*, 2022.

[5] S. Garg, P. Pundir, G. Rathee, P. K. Gupta, S. Garg, and S. Ahlawat, "On continuous integration / continuous delivery for automated deployment of machine learning models using MLOps," in *Proc. IEEE AIKE*, 2021.

[6] M.-A. Zöller and M. F. Huber, "Benchmark and survey of automated machine learning frameworks," *J. Artificial Intelligence Research*, vol. 70, pp. 409–472, 2021.

[7] L. Baier, T. Schlör, J. Schöffer, and N. Kühl, "Detecting concept drift with neural network model uncertainty," arXiv:2107.01873, 2021.

[8] B. Eck *et al.*, "A monitoring framework for deployed machine learning models with supply chain examples," in *Proc. IEEE Int. Conf. Big Data*, 2022.

[9] B. Eken, S. Pallewatta, N. K. Tran, A. Tosun, and M. A. Babar, "A multivocal review of MLOps practices, challenges and open issues," arXiv:2406.09737, 2024.

[10] X. He, K. Zhao, and X. Chu, "AutoML: A survey of the state-of-the-art," *Knowledge-Based Systems*, vol. 212, p. 106622, 2021.

[11] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," in *Proc. MLSys*, 2021.

[12] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM SDM*, 2007, pp. 443–448.

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[14] S. Rabanser, S. Günnemann, and Z. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Proc. NeurIPS*, 2019.

[15] E. K. Katalay, D. O. Dimandja, and J. F. Masakuna, "A multi-criteria automated MLOps pipeline for cost-effective cloud-based classifier retraining in response to data distribution shifts," arXiv:2512.11541, 2025.

[16] Amazon Web Services, "AWS free tier," https://aws.amazon.com/free/, 2026, accessed Nov. 2026.

[17] M. Harries, "SPLICE-2 comparative evaluation: Electricity pricing," University of New South Wales, Tech. Rep., 1999.